מעבדה 5

איתמר עמיר 314916354

ניתאי גילדור 318657780

.1

א. זמן המחזור שלנו הוא: T=1\f = 1000Ps. כעת נחשב לכל מסלול לפי הנוסחאות הבאות:

Data arrival time = T + logic time in path

Data required time = T – Tsetup

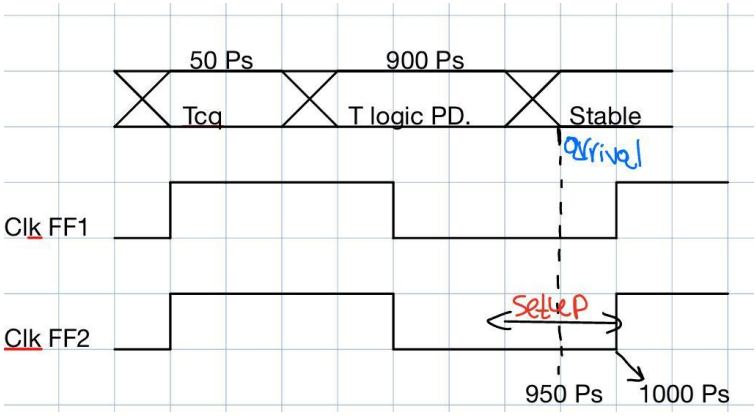
Slack : Tslack(setup) = Trequired_setup – Tarrival

Tslack(hold) = Tarrival – Trequired_hold

	FF1->FF2 setup	FF1->FF2 hold	FF3->FF2 setup	FF3->FF2 hold	FF2->FF3 setup	FF2->FF3 hold
Data arrival time	950	950	170	170	350	350
Data required time	900	150	900	150	900	150
Slack	-50	800	730	20	550	200
Meets time	no	yes	yes	Yes	Yes	Yes

ניתן לראות שהמסלול מ1 ל2 הוא Setup violation, עם slack של 50 ps , כלומר המידע מגיע מאוחר מדי על מנת להידגם על ידי FF2 בצורה תקינה.

שאר המסלולים עומדים בתנאי הTiming.



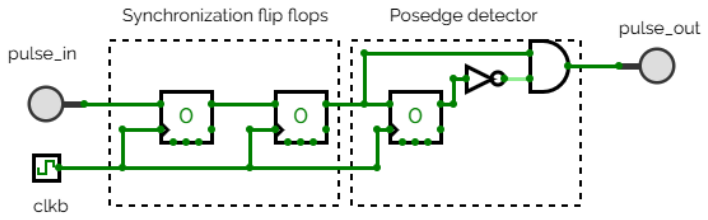
ב. מכיוון שגודל ההפרה הוא 50ps, נצטרך באפר על השעון של לפחות 50ps. כעת ייתווסף 50 במסלולים המסומנים בצהוב. החישוב של slack נשאר דומה מבחינת הנוסחא אך המספרים

משתנים. כעת יש לנו במסלול 3 ל2 Hold violation (באפר גדול יותר רק היה מחמיר את הבעיה)
אפשר להוסיף באפר בגודל 30 במסלול בין 3 ל2 (propagation delay)

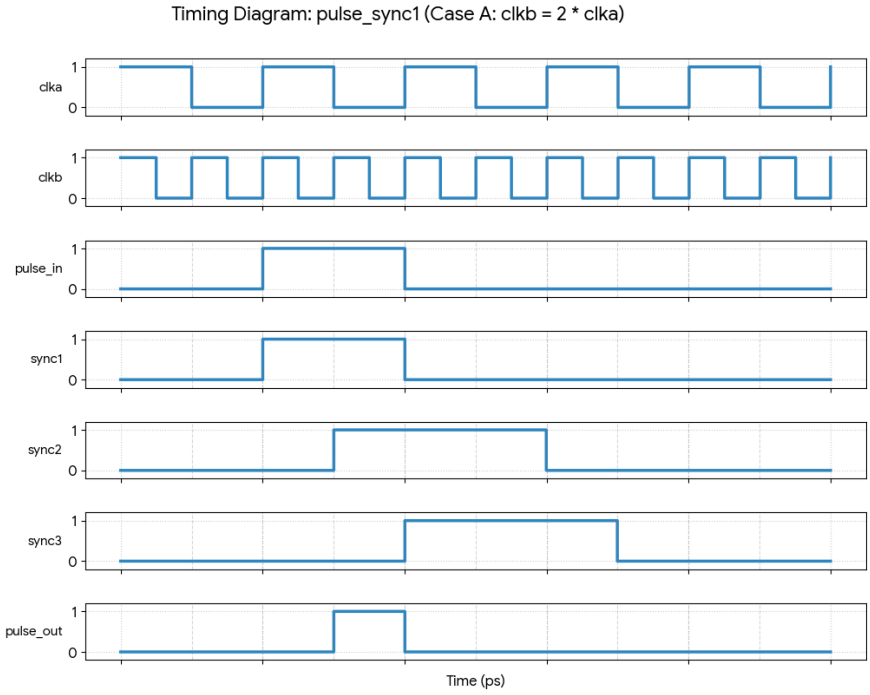
	FF1->FF2 setup	FF1->FF2 hold	FF3->FF2 setup	FF3->FF2 hold	FF2->FF3 setup	FF2->FF3 hold
Data arrival time	950	950	170	170	400	400(+50)
Data required time	950(+50)	200(+50)	950(+50)	200(+50)	900	150
Slack	0	750	780	-30	500	250
Meets time	yes	yes	yes	no	Yes	Yes

2. נפתור לכל אחד משלושת המצבים המתוארים. 3 המודולים תחת pulse_sync.sv.

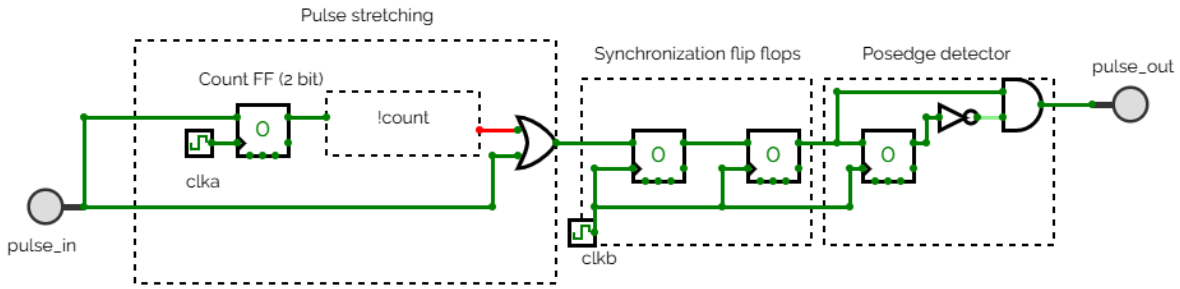
א. במקרה זה שעון B מהיר יותר משעון A פי 2. לכן דגימה בקצב של B בהכרח תתפוס את הפולס בקצב A. לכן נשתמש ב double flip flop synchronizer "רגיל", ובסוף נוסיף posedge detector, כך שייצא פולס באורך מחזור אחד של שעון B. אם לא נעשה זאת הפלט ב B יהיה ארוך מדי.
דיאגרמה:



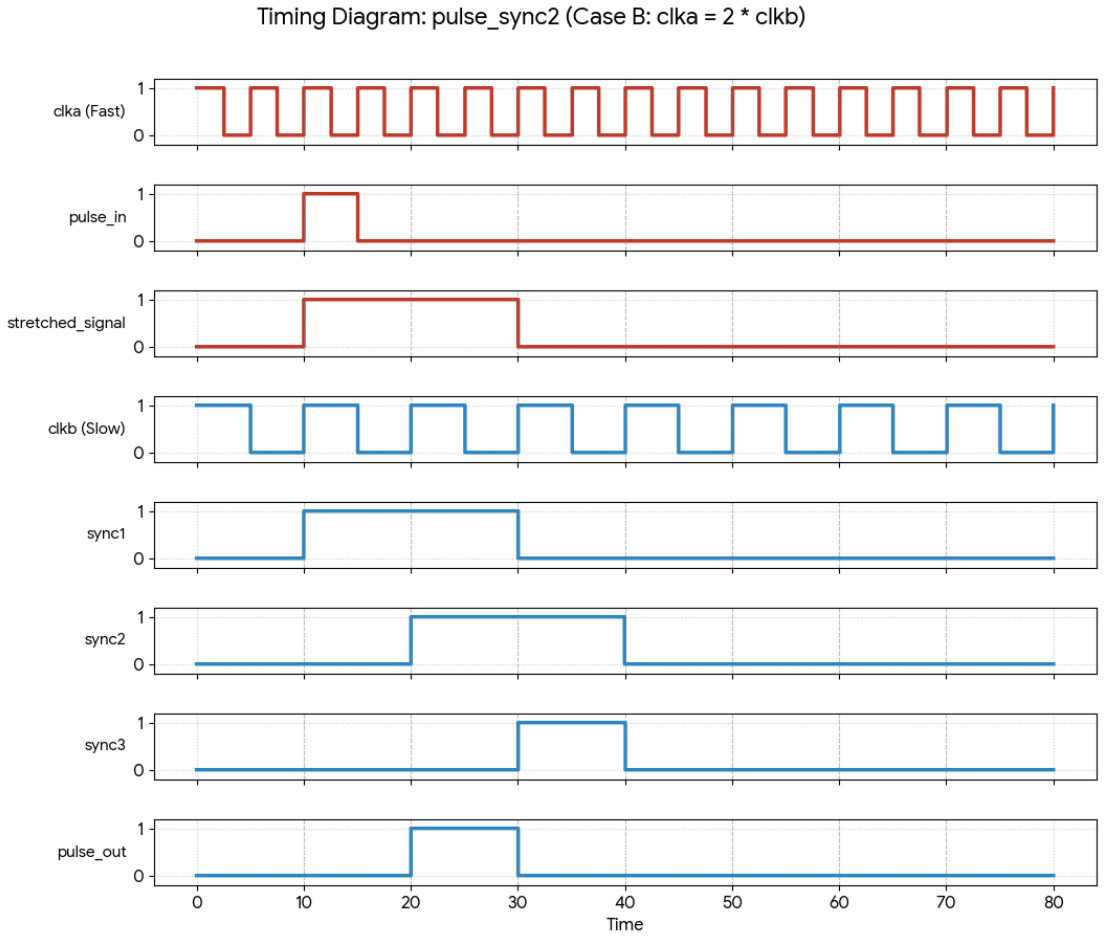
גלים (במקרה הזה הגלים מתואמים אבל זה יעבוד גם עם פאזה שונה):



ב. כעת בגלל ש CLKA מהיר יותר פי 2 מ B, יכול להיווצר מצב שבו בתחום B הפולס "מתפספס". לכן כמו שראינו בכתה יש צורך להגיל את אורך הפולס פי לפחות 1.5 מזמן המחזור של B. בנוסף למסנכרן כמו בסעיף הקודם כעת גם יש הארכת פולס בתחום A.
דיאגרמה:

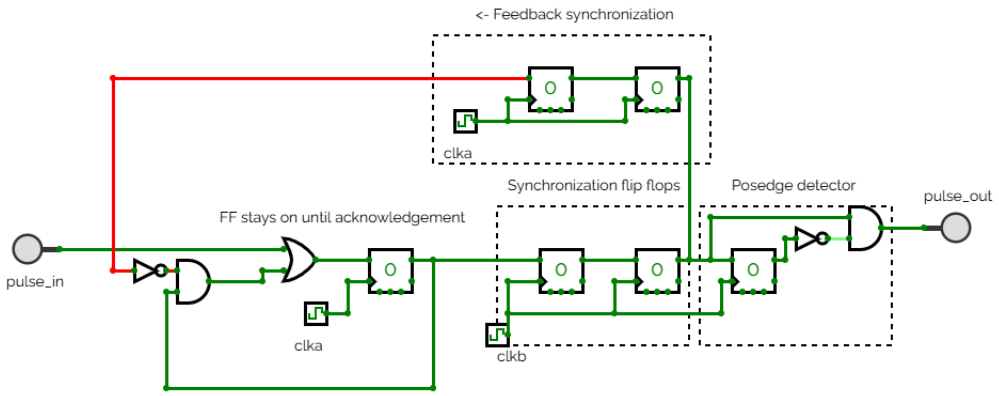


גלים:

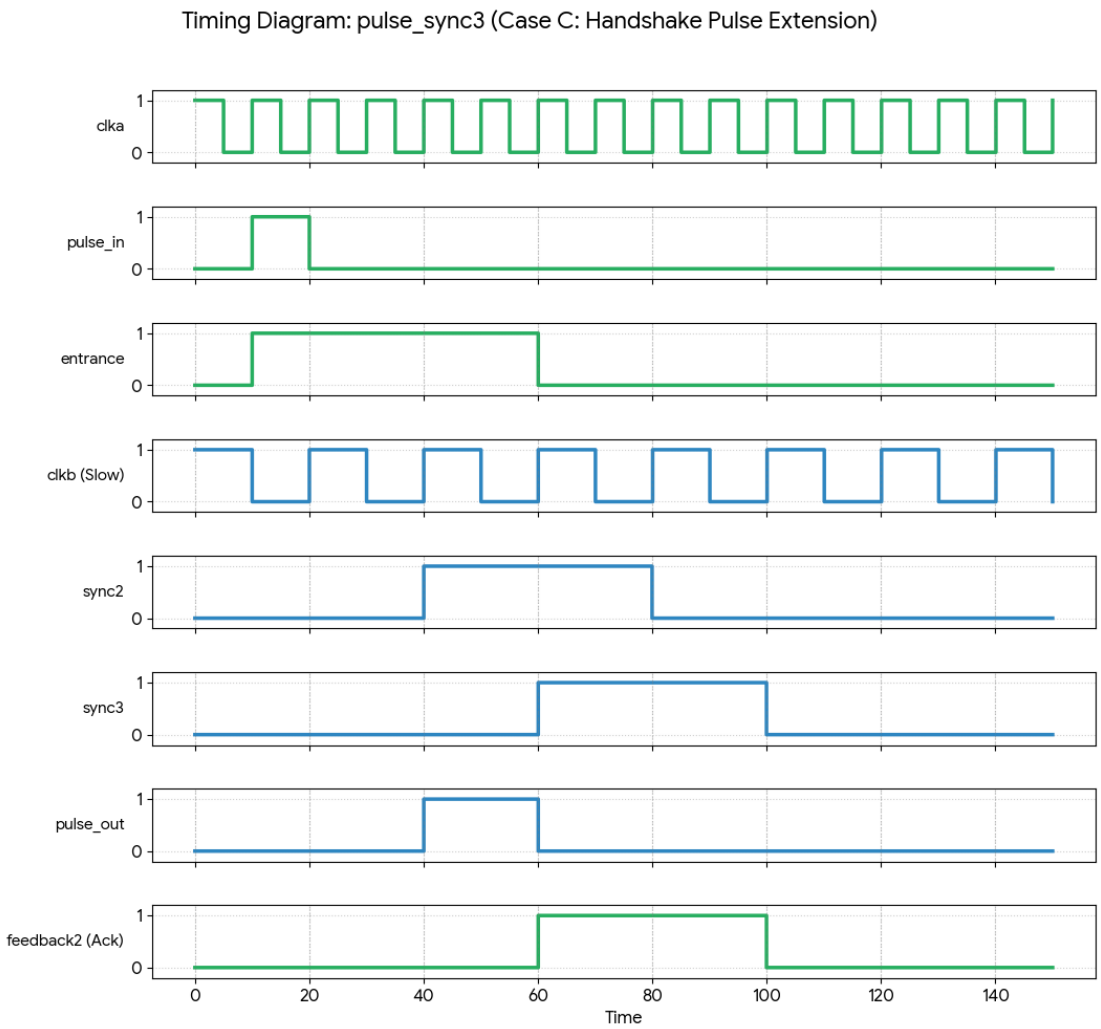


ג. כעת כאשר לא ידוע היחס בין השעונים נשתמש במנגנון שלא תלוי ביחס ביניהם: handshake mechanism.

דיאגרמה:

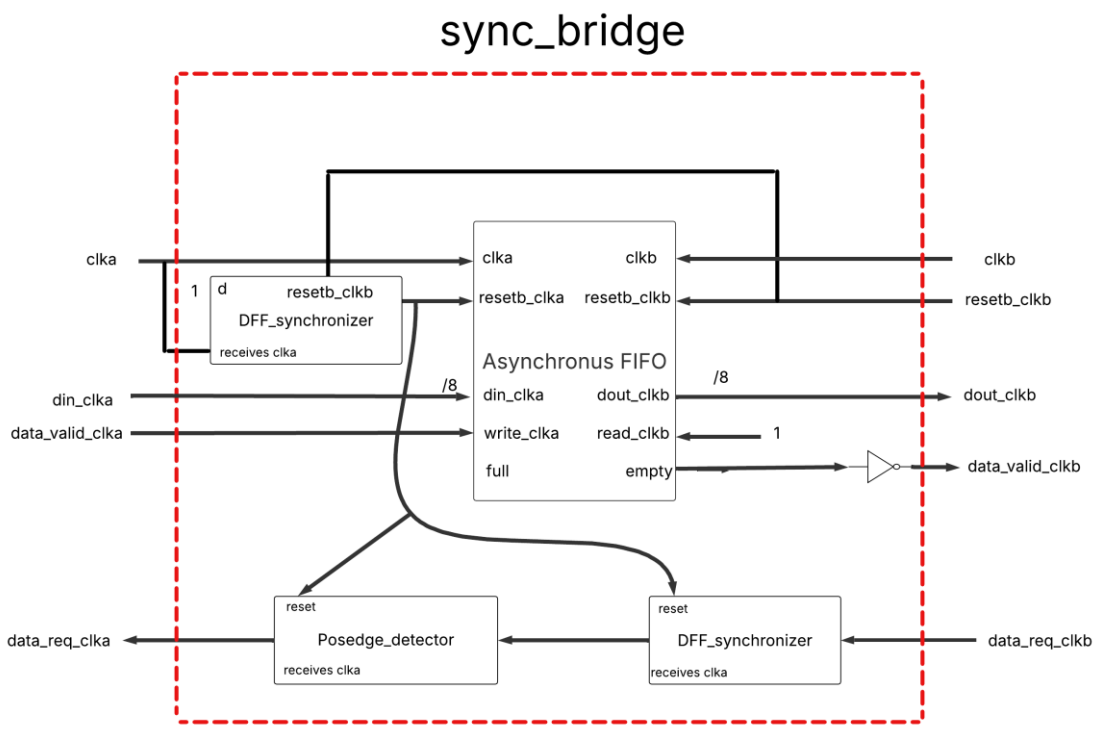


גלים:



.3

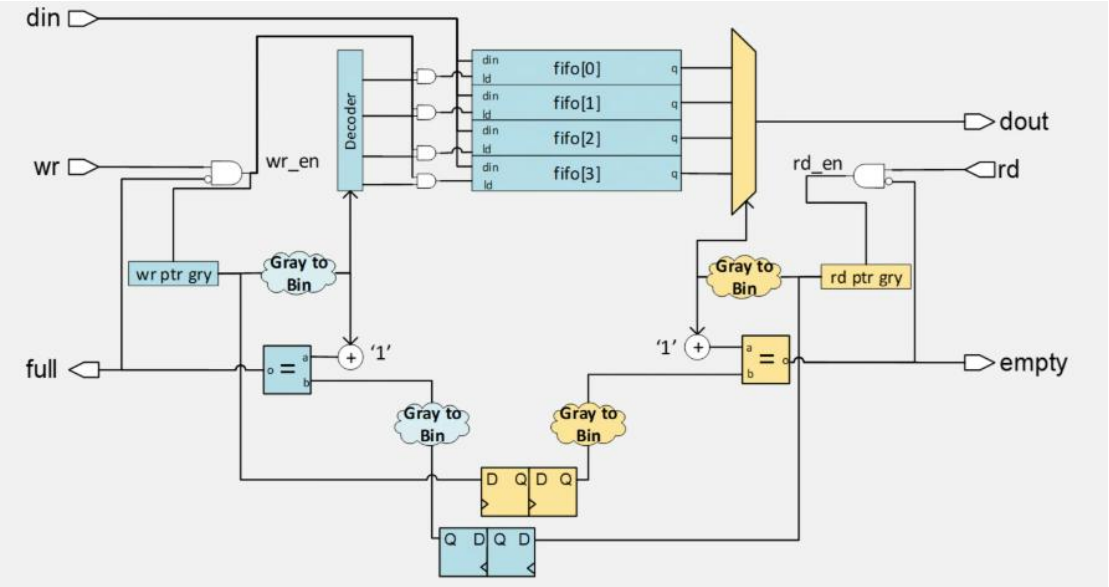
א. *הסברים נוספים על שיקולי התכנון נמצאים בסעיף ג
דיאגרמה מלאה:



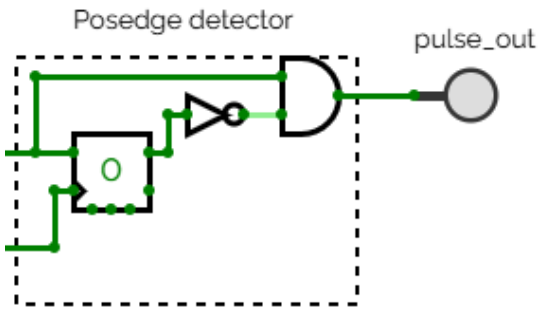
במרכז הדיאגרמה יושב asynchronous fifo שמימשנו כפי שראינו בכתה. מתחתיו יש DFF_synchronizer (הנתון מראש) ו posedge_detector. מטרתם היא כאשר יש data_req_clkb מבלוק B היא להמיר אותו למרחב התדר של A, ואז לדאוג שתהיה רק עליית שעון אחת במרחב A (בדומה לשאלה 2).

כל הבלוק sync_bridge מקבל רק את resetb_clkb, ולכן כל הריסטים בדיזיין מחוברים כך שעלייה של הריסט תהיה סנכרונית אך ירידה ואיפוס יהיו מיידיים לכל החלקים ולכן קיים גם בלוק DFF_synchronizer שמקבל 1 ב D ו RESET מ resetb_clkb.

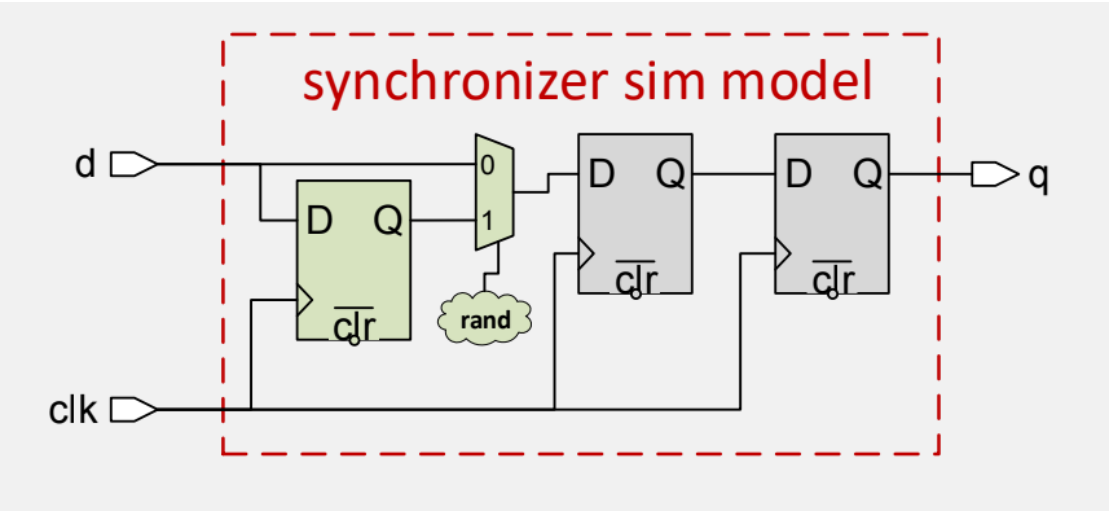
דיאגרמה של asynchronous_fifo כפי שראינו בכתה:



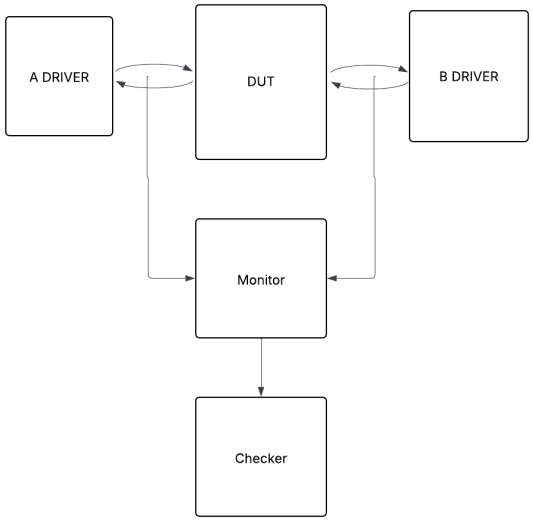
דיאגרמה של posedge_detector בדומה לשאלה 2:



דיאגרמה של dff_synchronizer כפי שראינו בכתה והשתמשנו בתרגיל (מהקובץ הנתון):



ב. דיאגרמה של sync_bridge_tb:



ראשית, הטסט בנץ' כולל DUT מסוג sync_bridge. בנוסף לכך קיימים דרייברים ל A ול B. דרייבר B מתחיל את הפעולות, על ידי שליחה של data_req_clkb. כעת באופן "מובנה" נבדק הDUT שהוא מעביר את ההוראה כמו שצריך ל A. לאחר מכן A שולח BURST ל DUT (כאשר A גם שומר את הערכים שהוא שלח בתוך QUEUE)

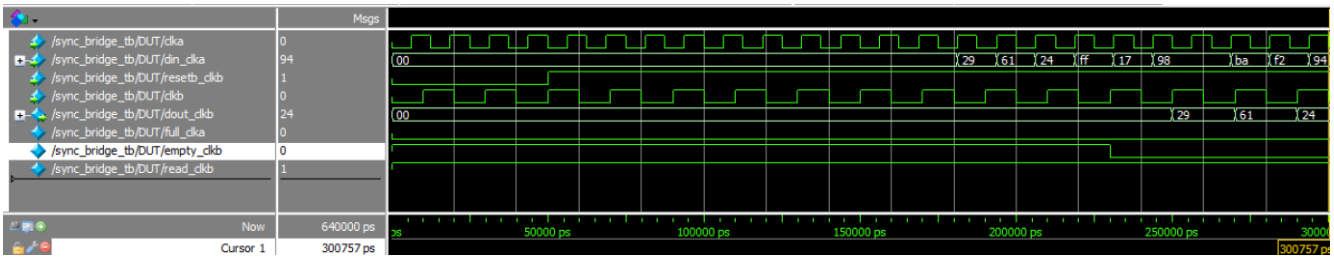
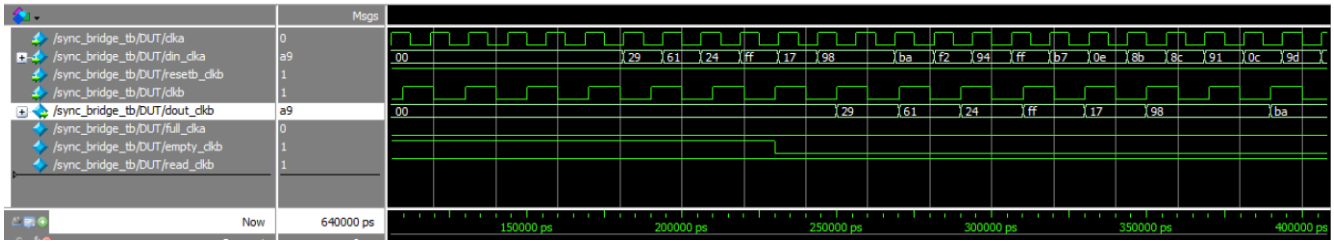
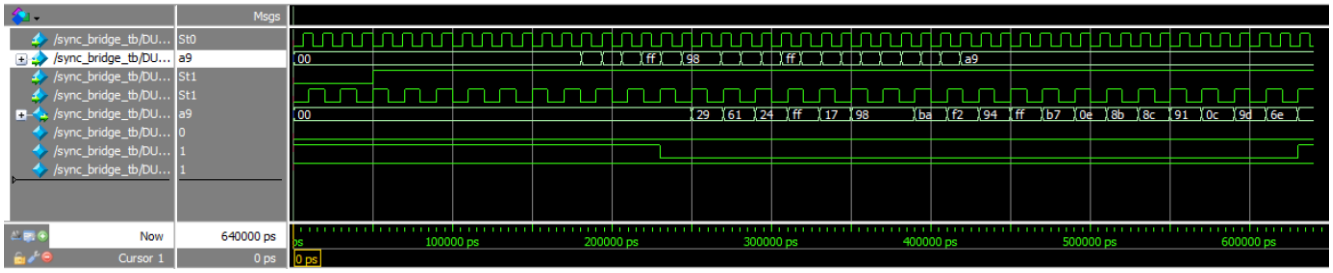
המוניטור והצ'קר יושבים על הממשקים בין A ל B. המוניטור אוסף את המידע ש"נקרא" ל B ואז משתמש בצ'קר בשביל להשוות בין הערך שנקרא לבין הערך שאמור לצאת לפי ה QUEUE.

ג. שיקולים בdesign :

(1) עומק ה FIFO – זמן המחזור של דומיין A הוא 12.5ns. לא צויין במעבדה אחרת ולכן אנחנו ממשים תחת ההנחה שכתיבה וקריאה היא רציפה ללא מחזורים "ריקים ביניהם (idle b write == 0 && idle b read == 0). זמן הכתיבה הכולל יהיה 20 (מספר המספרים ב BURST) כפול זמן מחזור של A, ולכן סה"כ 250ns.
כעת זמן המחזור של סייקל בודד של B הוא 20ns. נחלק את זמן הכתיבה הכולל בזמן מחזור יחיד, ונקבל 12.5. זאת אומרת, בזמן הכתיבה B ייקרא 12 מספרים מה FIFO, ולכן צריך לפחות 8 רגיסרטים למצב העמוס ביותר.
בגלל שלוקח זמן מתחילת הכתיבה עד תחילת הקריאה, נוסיף עוד 2 רגיסטרים. בנוסף בגלל מנגנון ה FULL-EMPTY במימוש זה של FIFO יוצא ש 2 רגיסטרים "מתבזבזים", ולכן נוסיף עוד 2. כעת יש צורך ב 12 רגיסטרים, אך בגלל שאנו משתמשים ב code grey לפויינטרים ואנחנו רוצים לשמור על התכונה הציקלית של הקוד גריי, נעגל למעלה לחזקה הבאה של 2, ולכן נשתמש ב **16 רגיסטרים**.

(2) RESET – sync_bridge מקבל רק את הריסט של B, אך יש בו אלמנטים שמסונכרנים לפי השעון של A. לכן, כל האלמנטים שמסונכרנים על שעון A מקבלים למעשה את אות מ DFF_synchronizer שמקבל ב D 1 ובריסט שלו מחובר לריסט B. המשמעות היא שכאשר B מחליט לעשות ריסט, כל רכיבי המערכת מתרסטים ביחד, אך כאשר ריסט עולה חזרה אז העלייה היא סינכרונית לפי תזמון השעון שכל חלק במערכת צריך.

(3) data_req- קיים מעבר מדומיין B לדומיין A ולכן צריך סינקרונייזר. בנוסף צריך שהאות שנקלט ב A יהיה רק בגודל סייקל אחד של A. לכן אשתמש ב dff_synchronizer ואחריו posedge_detector בדומה לשאלה 2.

ד. מצורפים קבצי הקוד המלאים. תיעוד של תוצאות ריצה:

```
VSIM 2> run -all
# Match: 00 at                250000
# Match: 29 at                270000
# Match: 61 at                290000
# Match: 24 at                310000
# Match: ff at                330000
# Match: 17 at                350000
# Match: 98 at                370000
# Match: 98 at                390000
# Match: ba at                410000
# Match: f2 at                430000
# Match: 94 at                450000
# Match: ff at                470000
# Match: b7 at                490000
# Match: 0e at                510000
# Match: 8b at                530000
# Match: 8c at                550000
# Match: 91 at                570000
# Match: 0c at                590000
# Match: 9d at                610000
# Match: 6e at                630000
# ** Note: $finish    : C:/intelFPGA/20.1/Lab5/sync_bridge_tb.sv(85)
#    Time: 640 ns  Iteration: 0  Instance: /sync_bridge_tb
# 1
```